



Chair of  
Information Theory  
and Data Analytics

**RWTH**AACHEN  
UNIVERSITY

---

---

# RESOURCE-EFFICIENT INFERENCE IN LARGE GENERATIVE MODELS

---

PATRICK TAZO KUETE

CHAIR OF INFORMATION THEORY AND DATA ANALYTICS  
RWTH AACHEN UNIVERSITY

---

---





Chair of  
Information Theory  
and Data Analytics

**RWTH**AACHEN  
UNIVERSITY

---

---

# RESOURCE-EFFICIENT INFERENCE IN LARGE GENERATIVE MODELS

---

PATRICK TAZO KUETE

A seminar submitted to the  
Faculty of Electrical Engineering and Information Technology,  
RWTH Aachen University,

reviewed by Univ.-Prof. Dr. Ing. Anke Schmeink and supervised by Hazem el Hadj Khalifa,  
M.Sc.

CHAIR OF INFORMATION THEORY AND DATA ANALYTICS  
RWTH AACHEN UNIVERSITY

---

---



## Eidesstattliche Versicherung

---

Name, Vorname

---

Matrikelnummer

Ich versichere hiermit an Eides Statt, dass ich die vorliegende Seminararbeit mit dem Titel

*Resource-Efficient Inference in Large Generative Models*

selbständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt. Für den Fall, dass die Arbeit zusätzlich auf einem Datenträger eingereicht wird, erkläre ich, dass die schriftliche und die elektronische Form vollständig übereinstimmen. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Aachen, den 18th June 2026

Ort, Datum

---

Unterschrift

### Belehrung:

#### **§ 156 StGB: Falsche Versicherung an Eides Statt**

Wer vor einer zur Abnahme einer Versicherung an Eides Statt zuständigen Behörde eine solche Versicherung falsch abgibt oder unter Berufung auf eine solche Versicherung falsch aussagt, wird mit Freiheitsstrafe bis zu drei Jahren oder mit Geldstrafe bestraft.

#### **§ 161 StGB: Fahrlässiger Falscheid; fahrlässige falsche Versicherung an Eides Statt**

(1) Wenn eine der in den §§ 154 bis 156 bezeichneten Handlungen aus Fahrlässigkeit begangen worden ist, so tritt Freiheitsstrafe bis zu einem Jahr oder Geldstrafe ein.

(2) Strafflosigkeit tritt ein, wenn der Täter die falsche Angabe rechtzeitig berichtigt. Die Vorschriften des § 158 Abs. 2 und 3 gelten entsprechend

Die vorstehende Belehrung habe ich zur Kenntnis genommen:

Aachen, den 18th June 2026

Ort, Datum

---

Unterschrift



## **Abstract**

A short summary.





## **Acknowledgements**

Thanks to the world!



# Contents

|                                                          |             |
|----------------------------------------------------------|-------------|
| <b>Acronyms</b>                                          | <b>xiii</b> |
| <b>Notation glossary</b>                                 | <b>xv</b>   |
| <b>1 Introduction</b>                                    | <b>1</b>    |
| 1.1 Motivation . . . . .                                 | 1           |
| 1.2 Aims . . . . .                                       | 1           |
| 1.3 Thesis structure . . . . .                           | 2           |
| <b>2 Theoretical foundations</b>                         | <b>3</b>    |
| 2.1 Large generative models . . . . .                    | 3           |
| 2.2 Inference challenges for large models . . . . .      | 3           |
| 2.3 Resource-efficient inference . . . . .               | 4           |
| <b>3 Resource-efficient inference methods</b>            | <b>5</b>    |
| 3.1 Model compression . . . . .                          | 5           |
| 3.1.1 Pruning . . . . .                                  | 5           |
| 3.1.2 Quantization . . . . .                             | 5           |
| 3.1.3 Knowledge distillation . . . . .                   | 6           |
| 3.2 Adaptive inference . . . . .                         | 6           |
| 3.2.1 Dynamic token processing . . . . .                 | 6           |
| 3.2.2 Conditional computation and sparsity . . . . .     | 6           |
| 3.3 Hardware-aware optimization . . . . .                | 7           |
| 3.4 Evaluation of resource-efficient inference . . . . . | 7           |
| <b>4 Conclusion and future work</b>                      | <b>9</b>    |
| <b>Bibliography</b>                                      | <b>11</b>   |



# Acronyms

**LLM** large language model 1, 2



# Notation glossary

|                                                                                                                                 |                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| $n \in \mathbb{N}$                                                                                                              | Natural number                                                                                                                  |
| $a \in \mathbb{R}$                                                                                                              | Real number, scalar                                                                                                             |
| $a \in \mathbb{R}_{\geq 0}$                                                                                                     | Non-negative real number, non-negative scalar                                                                                   |
| $\mathbf{a} \in \mathbb{R}^n$                                                                                                   | Real vector of size $n$                                                                                                         |
| $\mathbf{a} \in \mathbb{R}^{n \times m}$                                                                                        | Real matrix of size $n \times m$                                                                                                |
| $\mathfrak{P}(\mathbb{R}^n)$                                                                                                    | Set with all subsets of $\mathbb{R}^n$ as elements                                                                              |
| $:=$                                                                                                                            | Is defined as                                                                                                                   |
| $a_i, [\mathbf{a}]_i$                                                                                                           | $i$ th element in vector $\mathbf{a}$                                                                                           |
| $a_{ij}, [\mathbf{a}]_{ij}$                                                                                                     | Element at $i$ th row and $j$ th column in matrix $\mathbf{a}$                                                                  |
| $\mathbf{a}_{i:}, [\mathbf{a}]_{i:}$                                                                                            | $i$ th row in matrix $\mathbf{a}$                                                                                               |
| $\mathbf{a}_{:,j}, [\mathbf{a}]_{:,j}$                                                                                          | $j$ th column in matrix $\mathbf{a}$                                                                                            |
| $\mathbf{b} := \text{vec}(\mathbf{a}) \in \mathbb{R}^{nm}$                                                                      | Vector $\mathbf{b}$ contains all elements of $\mathbf{a}$ in row-wise order                                                     |
| $\emptyset$                                                                                                                     | Empty set                                                                                                                       |
| $\mathcal{I} \subset \mathbb{N}$                                                                                                | Set of indices                                                                                                                  |
| $ \mathcal{I} $                                                                                                                 | Cardinality of set $\mathcal{I}$                                                                                                |
| $\mathbf{b} := [\mathbf{a}]_{\mathcal{I}} \in \mathbb{R}^{ \mathcal{I} }$                                                       | Vector containing elements $\mathcal{I}$ of vector $\mathbf{a}$ with indices in ascending order                                 |
| $[\mathbf{a}]_{i\mathcal{I}} = [\mathbf{a}_{i:}]_{\mathcal{I}}, [\mathbf{a}]_{\mathcal{I}j} = [\mathbf{a}_{:,j}]_{\mathcal{I}}$ | Vector containing elements $\mathcal{I}$ of $i$ th row or $j$ th column respectively                                            |
| $\mathbf{b} := [\mathbf{a}]_{\mathcal{I}_1 \mathcal{I}_2} \in \mathbb{R}^{ \mathcal{I}_1  \times  \mathcal{I}_2 }$              | Matrix containing elements at all combinations of given row and column indices $\mathcal{I}_1$ and $\mathcal{I}_2$ respectively |
| $\mathbf{e}_i$                                                                                                                  | Unit vector $i$                                                                                                                 |
| $\mathbf{i}_n$                                                                                                                  | Identity matrix of size $n \times n$                                                                                            |
| $\mathbf{0}_n, \mathbf{0}_{n \times m}$                                                                                         | Zero-vector of size $n$ or zero-matrix of size $n \times m$                                                                     |

|                                         |                                                           |
|-----------------------------------------|-----------------------------------------------------------|
| $\mathbf{1}_n, \mathbf{1}_{n \times m}$ | One-vector of size $n$ or one-matrix of size $n \times m$ |
| $\mathbf{A}^\top$                       | Transposed matrix                                         |
| $\mathbf{A}^{-1}$                       | Matrix inverse                                            |
| $\mathbf{A}^\dagger$                    | Moore-Penrose pseudo-inverse                              |
| $\text{diag}(\mathbf{a})$               | Diagonal matrix with elements of $\mathbf{a}$             |
| $\text{diag}(\mathbf{a})$               | Diagonal matrix with diagonal elements of $\mathbf{a}$    |
| $ \cdot $                               | Absolute value                                            |
| $\ \cdot\ _2$                           | Euclidean norm                                            |
| $\ \cdot\ _F$                           | Frobenius norm                                            |
| $\ \cdot\ _0$                           | $l_0$ -norm, i.e., number of non-zero elements            |
| $[x]^+ = \max(x, 0)$                    | Positive part of $x$                                      |
| $[x]^- = \min(x, 0)$                    | Negative part of $x$                                      |
| $\neg a$                                | Logical NOT                                               |
| $a \wedge b$                            | Logical AND                                               |
| $a \vee b$                              | Logical OR                                                |
| $a \oplus b$                            | Logical XOR                                               |



# 1 Introduction

Large generative models have advanced the state of the art in natural language processing, code generation and multimodal reasoning. Since the transformer architecture was introduced, model capacity has grown from millions to hundreds of billions of parameters, and inference latency has become the primary bottleneck for practical deployment [4, 1].

## 1.1 Motivation

Large language models pose a challenge for deployment outside datacenters because inference requires both high memory bandwidth and large amounts of compute. For interactive applications and edge devices, the traditional strategy of using full-scale models is often infeasible. Resource-efficient inference therefore seeks a balanced trade-off: preserving model quality while reducing runtime cost, energy consumption and memory footprint [2, 3].

This seminar focuses on the techniques that enable inference for large language models (LLMs) beyond research prototypes. The motivation is to make these models accessible in resource-constrained environments such as mobile hardware, embedded systems and latency-critical services.

## 1.2 Aims

The main aims of this thesis are:

- Review the fundamental bottlenecks of large generative model inference and the role of transformer architectures in these costs.
- Analyze established efficiency techniques such as pruning, quantization and knowledge distillation, and explain how they reduce inference resource demands.

- Present system-level and algorithmic strategies that are relevant for deploying LLMs in realistic settings.
- Summarize the most promising directions for future research on resource-aware inference of generative models.

### 1.3 Thesis structure

This seminar is organized as follows. Chapter 2 presents the theoretical foundations of large generative models and the main sources of inference cost. Chapter 3 discusses resource-efficient inference methods, including compression and adaptive execution. Chapter 4 closes the paper with a summary of findings and recommendations for future work.

## 2 Theoretical foundations

### 2.1 Large generative models

Most modern large language models are based on the transformer architecture [4]. The transformer replaces recurrence with self-attention and a feed-forward network, and has become the dominant foundation for generative text models, including autoregressive decoders.

A single transformer block computes attention scores for an input sequence of length  $L$  and hidden dimension  $d$ . The self-attention operation has a complexity of  $\mathcal{O}(L^2d)$ , while the feed-forward network contributes  $\mathcal{O}(Ld^2)$ . As a result, inference cost grows rapidly with both sequence length and model dimension [4].

Large models with billions of parameters require substantial memory for model weights and intermediate activations. This challenge is significant for inference because transformer blocks must retain token representations across layers, and each layer adds to the total working memory.

### 2.2 Inference challenges for large models

Resource-efficient inference is concerned with reducing the practical cost of deploying large generative models. Three main resource categories are:

- **Compute:** the number of floating-point operations (FLOPs) executed during a forward pass.
- **Memory:** storage needed for model weights, activations and temporary buffers.
- **Latency and energy:** time to generate output and power consumed during inference.

Transformer-based generative models are especially demanding in these dimensions. The

total compute for a decoder-only model with  $N$  layers is roughly proportional to  $N(L^2d + Ld^2)$ , and the memory footprint is dominated by the weight matrices and attention context [1].

Scaling laws for language models show that larger models tend to generalize better, but they also demand more resources for inference [1]. For many applications, the goal is therefore not to maximize raw model size, but to maximize inference efficiency while maintaining acceptable task performance.

### 2.3 Resource-efficient inference

Resource-efficient inference aims to preserve the accuracy of a generative model while reducing its runtime requirements. The objective can be expressed informally as:

$$\text{minimize}_{\text{model, schedule}} \text{cost}_{\text{inference}} \quad \text{subject to} \quad \text{quality} \geq \text{threshold}.$$

A practical definition of cost depends on the deployment scenario. For mobile or edge systems, memory and energy are often the most important constraints. For cloud services, latency and throughput may be primary.

Because large models are already trained on abundant data, the most common efficiency methods are applied after or during training. These methods include model compression, adaptive inference strategies, and hardware-aware optimization. Each approach aims to reduce the effective inference burden without sacrificing the model’s generative capabilities.

## 3 Resource-efficient inference methods

This chapter reviews the most important techniques for reducing the cost of inference in large generative models. The methods are grouped into model compression, adaptive execution, and hardware-aware inference.

### 3.1 Model compression

Model compression reduces the effective size of a trained model while preserving task performance. The principal families of compression methods are pruning, quantization, and distillation.

#### 3.1.1 Pruning

Pruning removes weights or substructures that contribute little to the model’s output. Unstructured pruning eliminates individual parameters, producing sparse matrices that can be exploited by specialized sparse kernels. Structured pruning removes whole neurons, attention heads or transformer blocks, which is more amenable to efficient implementation on current hardware.

A class of methods known as movement pruning adapts sparsity during fine-tuning, using gradient signals to determine which weights should remain active [3]. This approach maintains accuracy by only removing weights that contribute little to the final loss.

#### 3.1.2 Quantization

Quantization replaces high-precision model weights and activations with lower-bit representations. Typical schemes use 8-bit integer quantization, but more aggressive formats such as 4-bit and mixed precision are now common for large transformers.

Quantization reduces memory footprint and increases throughput because smaller data types require less bandwidth and can be processed by faster hardware arithmetic units. For transformer inference, quantizing attention weights and feed-forward network matrices is particularly effective because these layers consume most of the compute budget.

#### 3.1.3 Knowledge distillation

Knowledge distillation transfers knowledge from a large teacher model into a smaller student model [2]. During training, the student learns to mimic the teacher’s output distribution, which often leads to improved performance compared to training the smaller model from scratch.

In the context of generative models, distillation can produce compact decoders that retain high-quality text generation while significantly lowering inference cost. Distilled models are thus a practical way to deploy resource-efficient LLMs on constrained hardware.

## 3.2 Adaptive inference

Adaptive inference methods change the amount of computation dynamically depending on the input or the current model state. These techniques are attractive because they focus resources where they are most needed.

#### 3.2.1 Dynamic token processing

For autoregressive generation, not every token requires the same amount of computation. Conditional computation mechanisms can adjust the depth or width of the network based on token importance. Early-exit architectures allow the model to stop processing a token after a few layers when the output is already confident, saving both compute and latency.

#### 3.2.2 Conditional computation and sparsity

Conditional computation activates only a subset of the model for each input. Examples include mixture-of-experts layers and sparse attention patterns. These techniques reduce actual runtime cost by avoiding full model evaluation, although they may require careful engineering to achieve hardware speedups.

### 3.3 Hardware-aware optimization

Hardware-aware optimization tailors inference to the capabilities of the target platform. This includes the use of mixed precision arithmetic, fused kernels, and execution optimizations provided by inference engines.

On modern accelerators, combining quantized weights with optimized matrix multiplication kernels can yield large speedups. For edge deployment, reducing memory access by using compact weight formats and reuse-friendly execution schedules is often more important than raw FLOP reduction.

### 3.4 Evaluation of resource-efficient inference

Practical evaluation must consider the target deployment scenario. Useful metrics include:

- Throughput (tokens per second)
- Latency per token or per prompt
- Memory footprint for weights and activations
- Model quality, typically measured via benchmark accuracy or likelihood

The best inference strategy depends on whether the goal is lower latency, reduced energy consumption, or smaller memory usage. For example, a compressed but still accurate model may be the best choice for mobile devices, while dynamic inference may be preferable when latency varies with input complexity.





## 4 Conclusion and future work

Resource-efficient inference for large generative models is a rapidly evolving field. The main strategies are model compression, adaptive inference and hardware-aware optimization.

Pruning, quantization and distillation each provide a way to decrease runtime cost while preserving model quality. These methods are especially useful when the deployment environment has fixed memory or compute limits.

Adaptive inference techniques such as conditional computation and early exits can further reduce average inference cost by adapting the computation to the input. Hardware-aware optimizations add another layer of efficiency by exploiting platform-specific features.

Future work should investigate how these methods can be combined in a unified inference pipeline. In particular, the interaction between low-bit quantization and dynamic execution remains an important research direction. Evaluating these techniques on real-world edge devices and latency-critical applications will be essential to understand their practical benefits.

In summary, resource-efficient inference is not a single solution but a collection of complementary methods. By carefully choosing compression, adaptation and hardware-aware techniques, it is possible to make large generative models more accessible and deployable outside datacenter-scale infrastructure.



# Bibliography

- [1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [2] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” in *Proceedings of the NIPS Deep Learning and Representation Learning Workshop*, 2015.
- [3] V. Sanh, T. Wolf, and A. M. Rush, “Movement pruning: Adaptive sparsity by fine-tuning,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 20 384–20 397, 2019.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, . Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.